NyayaTrace — treatment-aware precedent search for Indian case law

CSD358 Information Retrieval · Mid-term Hackathon · Track T6 — vertical search for law

Rakshit · Suvrat · Sumedha

Placeholder notice. Every figure typeset like `[LIKE_THIS]` is a measured result that is not in yet. They are filled from `runs/metrics.json`, `runs/efficiency.json` and `data/processed/extract_log.csv` before submission. No number in this report is estimated, rounded up from a guess, or carried over from a pilot run; an un-measured value is marked rather than invented, so that it cannot survive a read-through unnoticed.

1 Problem and track relevance

A lawyer looking for precedent is not asking "which document is about privacy". They are asking "which *binding* judgment, *still good law*, decided the point I am arguing, and how have later benches treated it". A general-purpose retrieval system answers the first question and is silent on the second — and the second is the one that decides a case.

Three properties of Indian Supreme Court judgments make this a *vertical* problem rather than a corpus to point an off-the-shelf index at.

The vocabulary is a citation system, not prose. The same provision appears as *Section 302 IPC*, *S. 302 of the Indian Penal Code, 1860*, and *u/s 302*. To a standard tokenizer those three strings share almost no terms, so one provision is scattered across three postings lists and a query for any one of them misses the other two. The problem is live rather than historical: the Penal Code was replaced by the Bharatiya Nyaya Sanhita on 1 July 2024 and the section numbers moved, so a corpus spanning 1950–2025 contains both numbering systems for the same offence.

A judgment is a structured document. It has a cause title, parties, a bench, the statutes it turns on, an operative order, and a very long body of facts, submissions and reasoning. Roughly `[BODY_SHARE_PCT]` of the tokens we index sit in the body zone. A flat bag-of-words model therefore spends almost all of its evidence on recital, and the holding — the part a lawyer actually wants — is drowned by it. The bench, meanwhile, is not evidence of relevance at all: it is a *filter*, because a five-judge bench binds a three-judge bench.

Citation count and reliability come apart. A judgment can be cited hundreds of times and still be the one case you must not cite, because a later and larger bench overruled it. Every earlier citation still points at it, and no link-popularity measure ever retracts a vote. Text relevance cannot detect this, and neither can plain PageRank; both will rank a dead precedent at the top.

NyayaTrace is built around these three. It is squarely Track T6 because every component exploits domain structure a general web engine has no access to: zones derived from the shape of a judgment, parametric metadata (year, bench size, judges, acts, case type) that supports real filters, a citation graph whose edges carry a *legal* relation rather than a hyperlink, and an authority score that is a statement about precedential standing rather than about popularity.

Corpus. We use the AWS Open Data release of Indian Supreme Court judgments (CC-BY-4.0). After de-duplicating on the bucket path there are **42,371** English judgments spanning **1950–2025**. All 42,371 are nodes of the citation graph and the source of parametric facts. We index the *text* of a stratified sample of at most **200 judgments per year** (seed 358, so the draw is reproducible), taking years with fewer judgments whole. This gives **≈14,000** indexed documents with equal temporal coverage, so the whole history of the Court is searchable rather than only the last decade. The sampling rate is a known limitation and we treat it as one — see §6.

2 How we used IR

The system is one code path with switchable layers. Figure 1 shows the four bands: what runs at build time, what sits on disk, what happens when a query arrives, and what the user is finally shown.

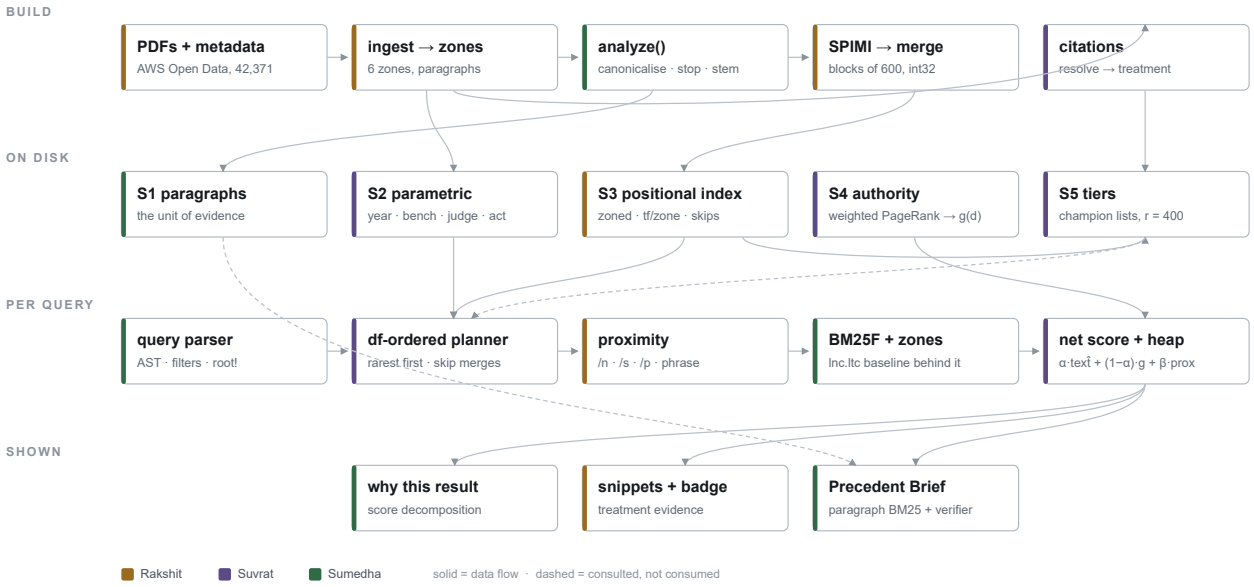


Figure 1 — the NyayaTrace pipeline. The build row runs once; the stores row is what exists on disk (S1–S5 of the team contract); the query row is what executes per search; the bottom row is what the user is shown. Colour is ownership, which is also how the work was divided (§7). Note that `analyze()` is the only path into the index *and* the only path into a query, which is the invariant the whole system is built on.

2.1 Ingest and zoning

PDFs and the metadata parquet files are pulled from the bucket, text is extracted page by page, and a page yielding fewer than 200 words is treated as a scanned image, with the document logged as a drop rather than silently kept. Every retained judgment is cut into the six zones of `config.ZONES` — `title`, `parties`, `bench`, `statutes`, `order`, `body` — and into numbered paragraphs, which are the unit of evidence for snippets, proximity and the Precedent Brief. Each attempted document gets a row in `extract_log.csv` with its drop reason, so the corpus is auditable rather than merely large.

2.2 The analyzer

`analyze(text, zone, para_id)` is the single entry point for documents **and** queries. That is the one invariant we froze before writing anything else: if the two ever diverge, matching breaks silently and no test catches it. It case-folds, tokenises, canonicalises statutes (§3.1), drops a legal stop list — words like *appellant*, *hon'ble*, *judgment* carry no discriminative power when every document contains them — and applies a Porter stem with statute tokens exempt, because `ipc§302` is an identifier and stemming an identifier is a bug. Each emitted token carries its surface form, its running offset within the zone, its sentence id and its paragraph id.

2.3 Zoned positional inverted index

Built with SPIMI: 600 documents are inverted in memory, flushed to a block on disk, and the blocks merged once into a single memory-mapped `int32` array. Nothing holds the whole index in RAM and nothing is a Python dictionary at query time. A posting is `(doc_id, {zone → tf}, positions)` — which document, how often in each zone, and exactly where. Every ⌈√df⌉ entries the list carries a skip pointer, so a merge between a short list and a long one does not read the long one end to end. The index also stores `doc_len`, `zone_len` per document and the `lnc` vector norms the cosine baseline needs. Sizes: `[VOCAB]` distinct terms, `[TOKENS]` tokens, `[POSTINGS_MB]` MB of postings, built in `[BUILD_MIN]` minutes.

2.4 Parametric index

A separate store over year, bench size, judge names, acts, case type and disposal, exposing `select(field, op, value) → sorted doc ids`. This is what makes `bench:>=5` and `year:>=2000` genuine constraints rather than keyword matches against the bench zone, and it is why the bench zone has weight 0 in ranking: scoring it as well would be double counting.

2.5 Query language and parser

A Westlaw-flavoured syntax the profession already knows: `AND OR NOT`, quoted phrases, `/n` (within n words), `/s` (same sentence), `/p` (same paragraph), root expansion `bail!` by dictionary prefix scan, zone restriction `title:(pri-`

`vacy)`, and filters `year:>=2000 bench:>=5 judge:chandrachud act:IPC`. The parser returns an AST of `TermNode / PhraseNode / ProxNode / AndNode / OrNode / NotNode / FilterNode`, plus — for a query with no operators — a plain bag of terms that takes the free-text fast path.

2.6 Boolean planning and merges

The planner never executes a conjunction in the order the user typed it. It orders the operands by document frequency and processes the rarest first, because an intersection can only shrink: starting small keeps every later merge small. Selective parametric filters are folded in first for the same reason. All intersections use the skip pointers. Index elimination drops query terms below an idf of 0.35, and for a free-text query of four or more terms only documents matching at least half of them are scored at all.

2.7 Proximity

Because every token carries its offset, sentence id and paragraph id, `/n`, `/s` and `/p` are *one* two-pointer merge over two position lists with three different comparisons: $|pos_a - pos_b| \le k$, $sent_a == sent_b$, $para_a == para_b$. A quoted phrase is the ordered, tightest case of the same walk. The merge is linear in the two lists and never materialises a cross-product.

2.8 Ranking, as switchable layers

1. **lnc.ltc cosine** — the textbook tf-idf baseline: flat document, no zones.
2. **Weighted zone scoring** — a per-zone score combined with `title 3.0, statutes 2.0, order 1.5, body 1.0, parties 0.5, bench 0`.
3. **BM25F** — the per-zone term frequencies are combined *before* saturation, each with its own length normaliser b_z, and the sum is then saturated with $k_1 = 1.2$. Saturating per zone and adding afterwards would let a long body outshout a title, which is the whole reason BM25F exists rather than BM25 over a concatenated document.
4. **Proximity contribution** — the tightest window the operators found, inverted.
5. **Net score** — $net = \alpha \cdot \hat{text} + (1-\alpha) \cdot g(d) + \beta \cdot prox$ with $\alpha = 0.80$, $\beta = 0.05$, where $\hat{text}$ is min–max normalised within the candidate set and $g(d)$ is the static authority of §3.4.

Setting $\alpha = 1$ removes authority; swapping the scorer returns the cosine baseline. Each of the six evaluated systems is one flag away from its neighbour.

2.9 Efficiency structures

Champion lists hold the top 400 documents per term by a static quality measure, and a two-tier index serves the free-text fast path from tier 1, falling back to the full postings only when tier 1 cannot fill *k*. Results are collected through a heap rather than a full sort. Measured effect in §5.4.

2.10 Explanation

Every result carries a `ScoreExplanation` dataclass: raw and normalised text score, per-zone contributions, top terms by idf·tf, proximity window, `g(d)` and its `pagerank` / `bench` parts, the status penalty, α, β and the net. Because it is a dataclass produced by the scorer rather than a string assembled for display, it cannot drift away from what was actually computed.

3 Beyond IR

Five components are not retrieval techniques; they are domain engineering that retrieval then exploits.

3.1 Statute canonicalisation and the IPC↔BNS crosswalk

A grammar over Indian statutory citation forms — not a phrase list — recognises `Section 302 IPC`, `S. 302 of the Indian Penal Code, 1860`, `u/s 302`, `sec. 302 I.P.C.` and their punctuation variants, and rewrites all of them to the single token `ipc§302`. It covers the Constitution, IPC, CrPC, CPC, the Evidence Act and the three 2023 replacement codes, and handles sub-sections and clauses. The rewrite happens once at build time, so it costs nothing per query. On the query side a crosswalk expands `ipc§302` to also fire `bns§103` and vice versa, because the corpus straddles the 2024 recodification; the expansion is a flag, so the S1 ablation can measure what canonicalisation alone is worth.

3.2 Citation extraction and resolution

Judgments cite each other in prose. We extract candidate citations and resolve them to node ids in three passes: neutral citation (`2017 INSC 1136`), SCR citation (`[2017] 11 S.C.R. 1029`), and case-name matching requiring a Jaccard

overlap of at least 0.6 over normalised party-name tokens. Each edge records how it was resolved, so an unresolved or name-matched edge can be excluded from an ablation. Result: `[EDGES]` edges, `[RESOLVED_PCT]` resolved.

3.3 Treatment classification

The novel part. For each citation we take the sentence and the surrounding paragraph and classify the *treatment* the citing court applied, with a cue-phrase classifier over four classes: **followed** ("we respectfully follow", "we are in agreement with"), **distinguished** ("is distinguishable on facts", "turned on its own facts"), **overruled** ("stands overruled", "no longer good law", "is hereby overruled") and **mentioned** (everything else). This is a legal-reasoning label, not a link type, and it is what makes the authority model possible.

3.4 Treatment-weighted authority

The citation graph is weighted by treatment — `followed 1.0, mentioned 0.6, distinguished 0.3, overruled 0.0` — row-normalised, and we run our own power iteration (`λ = 0.85`, 60 iterations, tolerance 1e-10; no graph library). An overruling edge therefore transfers no authority at all. The PageRank is then combined with bench size, which is the Court's own hierarchy of precedential weight, and multiplied by a status penalty:

```
g(d)    = (0.75 · pagerank_norm + 0.25 · bench_norm) × penalty
penalty = 0.25  if overruled
          0.70  if frequently distinguished
          1.00  otherwise
```

A case counts as *frequently distinguished* when at least 34% of its in-edges are distinguishing and it has at least three such edges. `[N_OVERRULED]` judgments in our corpus carry an overruled status; each is flagged in the interface with the citing judgment and the paragraph that did it.

3.5 Precedent Brief — retrieval-augmented generation without vectors

An optional feature that drafts a short brief. Paragraph-level BM25 over the same positional index selects at most 8 paragraphs from the top 5 documents, never more than 2 from one judgment. A language model is then required to attach a paragraph id to every sentence it writes. A tf-idf cosine verifier re-tokenises each generated sentence through `analyze()` and scores it against the paragraph it claims; below a cosine of 0.20 the sentence is flagged **unsupported** in the output rather than deleted, so the reader sees the disagreement. There is no embedding model and no vector store anywhere in the system: the retrieval *is* the system we already measured.

4 Novelty and creativity

Treatment-aware authority is the main claim. Link analysis over a citation graph is standard; treating the *label on the edge* as the weight, and separating "how much authority flows" from "is this still good law", is not. The two are deliberately different mechanisms: an overruling edge carries weight 0 in the PageRank (it transfers nothing) *and* triggers a multiplicative status penalty on the target (it demotes). A purely spectral method cannot express the second, because a node with no outgoing authority is not the same thing as a node that must be pushed down the ranking. The visible consequence is the one a practitioner cares about: the most-cited judgment on a point can be demoted and badged, with the sentence that overruled it shown as evidence.

Statute tokens as first-class index terms. Collapsing three surface forms to `ipc§302` turns a vocabulary-mismatch problem into an exact-match problem, and makes `act:IPC` and `statutes:(ipc§302)` meaningful. The IPC↔BNS crosswalk makes the index robust to a statutory change that happened in the middle of the corpus — a problem that simply does not arise in web search.

Explanation as a contract, not a feature. `ScoreExplanation` is part of the frozen interface in `nyaya/contracts.py`. Every result decomposes into per-zone contributions, top terms, proximity, authority and penalty. That is what let three people build against each other's components in parallel, and debug ranking disagreements by reading a decomposition instead of guessing.

The ablation ladder is a flag, not six implementations. B0 → S1 → S2 → S2′ → S3 → S4 differ by one constant or one branch each. That is a design decision with a cost — the code carries a baseline it will never ship — and the payoff is that every row of §5 is attributable to exactly one idea.

Vectorless RAG with a verifier. The brief reuses the index we already built and measured, and is wrapped so the model cannot assert anything the corpus does not contain. In a domain where a confidently fabricated citation is a professional hazard, flagging unsupported sentences is more useful than hiding them.

5 Evaluation

Everything is measured through one entry point, `search(query, k, mode)`, so the six systems are genuinely comparable.

Table 1 — the six systems. Each is one flag from its neighbour.

Mode	What it is
B0	flat lnc.ltc tf-idf, plain tokenizer, no zones, no authority
S1	B0 + statute canonicalisation + legal stop list
S2	S1 + weighted zone scoring
S2′	S1 + BM25F instead of cosine
S3	S2′ + authority from plain (unweighted) PageRank
S4	S3 + treatment-aware `g(d)` + proximity — the full system

5.1 Judged queries

`[N_QUERIES]` hand-written queries spanning constitutional, criminal, service and commercial matters, each with a stated information need. Results from all six systems were pooled to depth `[POOL_DEPTH]`, and `[N_JUDGED]` judgments were graded 0–3 by a law-reading member of the team who could not see which system had retrieved what. The qrels and the query set live in `eval/queries/`.

Table 2 — retrieval effectiveness over the judged query set.

Metric	B0	S1	S2	S2′	S3	S4
P@5	`[B0_P5]`	`[S1_P5]`	`[S2_P5]`	`[S2P_P5]`	`[S3_P5]`	`[S4_P5]`
P@10	`[B0_P10]`	`[S1_P10]`	`[S2_P10]`	`[S2P_P10]`	`[S3_P10]`	`[S4_P10]`
R@20	`[B0_R20]`	`[S1_R20]`	`[S2_R20]`	`[S2P_R20]`	`[S3_R20]`	`[S4_R20]`
MAP	`[B0_MAP]`	`[S1_MAP]`	`[S2_MAP]`	`[S2P_MAP]`	`[S3_MAP]`	`[S4_MAP]`
nDCG@10	`[B0_NDCG]`	`[S1_NDCG]`	`[S2_NDCG]`	`[S2P_NDCG]`	`[S3_NDCG]`	`[S4_NDCG]`

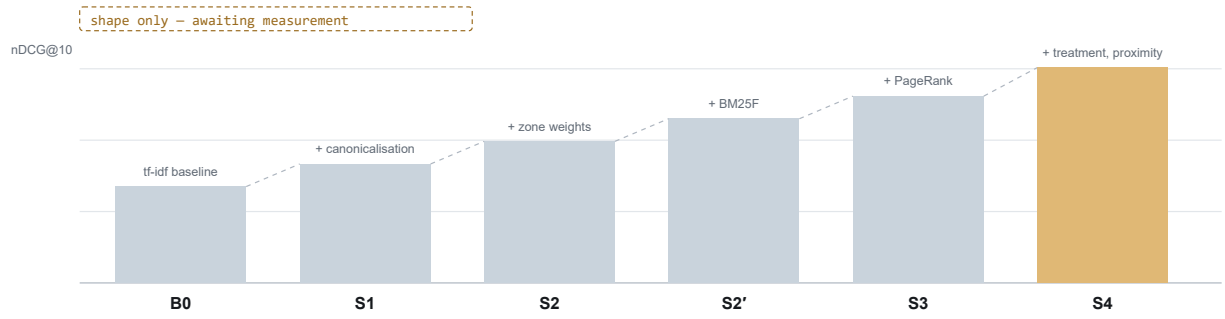


Figure 2 — the ablation ladder. Bar *heights are placeholders* and carry no information until the harness has run; the figure is here to fix the reading, not to assert a result. Each step is annotated with the single change that produced it, and the final chart will keep that annotation so a step that fails to move is as visible as one that does.

We report the shape of the ladder, including the steps that do not move. An ablation that fails to pay for itself is a result, and a test set of this size will not resolve small differences; we will say which gaps we consider real and which we do not.

5.2 Citation recovery — an automatic track with no human judge

Take an indexed judgment, hide every case it actually cites, use its own holding paragraphs as the query, and ask how many of the hidden citations come back. Over `[CR_N]` such queries: R@20 `[CR_R20]`, MRR `[CR_MRR]`. The ceiling here is not 1.0 and cannot be: a cited precedent is only retrievable if it survived the ≤200-per-year sample, so this number measures our ranking *and* our sampling together. We report it with that caveat attached rather than quietly excluding unsampled targets.

5.3 Operator correctness

Every operator family — phrase, `/n`, `/s`, `/p`, `AND` / `OR` / `NOT`, `root!`, filters, zone restriction — is checked against a brute-force linear scan of the raw text over the same corpus. The two result sets must be *identical*, not merely similar.

Over [OP_N] cases: [OP_AGREE] set agreement. This is pass-or-fail, and is the one number in the project that is allowed to be boring.

5.4 Efficiency

Table 3 — free-text fast path, median over the query set.

Configuration	Latency
score every posting	[MS_FULL] ms
champion lists (r = 400) + tier 1	[MS_TIER] ms
speed-up, for [TIER_RECALL_LOSS] of top-10 recall given up	[SPEEDUP]

Skip-pointer merges touch [SKIP_GAIN] fewer postings than a linear walk of the same lists. Median end-to-end query latency is [MS_MEDIAN] ms.

5.5 Precedent Brief verification

[BR_SENT] generated sentences, [BR_CITED_PCT] carrying a paragraph id, [BR_FLAGGED_PCT] flagged unsupported by the cosine verifier at threshold 0.20.

6 Limitations and next steps

The sample caps recall, and we do not hide it. We index ≈14,000 of 42,371 judgments. A precedent that was not sampled cannot be retrieved, which puts a hard ceiling on §5.2 and depresses recall on any query whose best answer happens to fall outside the draw. The citation *graph* covers all 42,371 nodes, so authority is computed over the full universe even where we lack the text — but that also means some high-authority nodes are unsearchable.

The treatment classifier cannot read a denial. It matches surface cues, so "We are *unable to hold* that the decision in *Rajesh v. State stands overruled*" contains the cue for an overruling inside an explicit refusal to overrule, and is mis-labelled. A case that was upheld then loses three quarters of its `g(d)`. On our hand-labelled sample of [TREAT_N] edges the classifier is right [TREAT_ACC] of the time, and [NEG_ERR] of its errors are negated, conditional or hypothetical cues of exactly this kind. The fix is a negation and modality *scope* around the cue, not a longer phrase list; until it lands, the flag is visible in the interface together with the sentence that caused it, so a user can overrule the system.

Name-based citation resolution is the weakest link in the graph. Neutral and SCR citations resolve cleanly; a Jaccard threshold over party names does not, and Indian case names repeat heavily (*State of Maharashtra v. …* appears constantly). Every edge records `resolved_by`, so the damage is measurable, but it is real.

Extraction loss. Older judgments are scans. Pages under 200 words are treated as images and the document is dropped, which biases the indexed corpus towards the typeset era within each year's quota.

One judge, one assessor pool. The relevance judgments come from one person on the team. They were made blind to system identity, but they are not independent, and [N_QUERIES] queries is a small test set. Differences of a few points in §5.1 should not be read as real.

No High Courts, no statutes as documents. Precedent in practice runs through High Court decisions and the bare text of the statutes themselves; we index neither.

Table 4 — roadmap.

Horizon	Work	Why it is next
Next	Negation and modality scope around treatment cues: a small dependency window, so "unable to hold" cancels the match	The single largest source of wrong labels — and a wrong label is worse than a missing one, because it demotes good law
Next	Per-query confidence intervals and a significance test over the judged set	Several §5.1 gaps are probably not real; saying so is cheap and honest
Then	Replace the cue classifier with a supervised model over the hand-labelled set, keeping the cues as features	Turns our test set into training data, with a fresh held-out set
Then	Lift the per-year cap for the years the query set actually touches, then for everything	Directly raises the §5.2 ceiling
After	Index High Court judgments and bare statutes; add a *cited-by* browse view over the treatment graph	The graph is already the right shape for it; this is what makes it a tool rather than a demo
After	Learned ranking over the existing features (zone scores, proximity, `g(d)`, bench, recency) once the judged set can support it	Our α and β are hand-set; they should be fit, not chosen

7 Work division

Interfaces were frozen at kick-off in `nyaya/contracts.py`, together with the constants in `nyaya/config.py`, and the three of us built against them in parallel. Ownership is file-level: nobody edits a module they do not own.

Table 5 — who built what.

	Rakshit (M1)	Suvrat (M2)	Sumedha (M3)
Corpus	download, PDF extraction, zoning, paragraph segmentation, extract log	metadata join, universe and manifest, stratified sample	—
Index	SPIMI build, zoned positional postings, skip pointers, memory-mapped store (S3)	parametric index (S2), champion lists and tiers (S5)	paragraph store (S1), collection statistics
Query	proximity operators `/n` `/s` `/p`, phrase matching	df-ordered Boolean planner, index elimination, filter folding	query language, parser, normaliser, root expansion
Ranking	proximity contribution, snippets	citation graph, treatment-weighted PageRank, `g(d)`, net score, heap	lnc.ltc cosine, weighted zone scoring, BM25F, "why this result"
Beyond IR	cue-phrase treatment classifier	citation extraction and resolution	statute canonicaliser, IPC↔BNS crosswalk, Precedent Brief and verifier
Evaluation	evaluation harness, metrics (P@k, R@k, MAP, nDCG), relevance judging	citation-recovery and efficiency tracks	query-set curation, operator correctness tests
Delivery	video production	report assembly	CLI and interface, diagrams and script

Shared and append-only: `config.py`, `contracts.py`, `utils.py`, `README.md`. Every component has at least one readable test in `tests/`, and every component can print an intermediate output on demand — postings, plan, weights, scores — which is both a debugging contract and what the demo video shows.

AI-use declaration

This project was built with AI coding agents. The assignment permits that on condition it is declared, so it is declared here specifically.

What the AI did

Most of the implementation code in `nyaya/`, the tests in `tests/`, the Remotion video project in `video/`, and the first draft of this report were written by **Claude Code (Anthropic)** agents working under our direction, in parallel, each confined to the files its owner is responsible for.

What we did

We chose the problem and the track. We designed the architecture: the six zones and their weights, the mode ladder B0→S4, the treatment-weighted authority model and the decision to separate edge weight from status penalty, the net-score formula, and the decision to build vectorless RAG with a verifier rather than add an embedding store. We wrote and froze the interface contracts in `nyaya/contracts.py` and the constants in `nyaya/config.py` before any implementation began, and the agents were required to build against them. We set the corpus policy — the universe/indexed-subset split, the ≤200-per-year stratified sample, the seed — and accepted its cost to recall. We designed the evaluation: which systems to compare, which metrics, pooled judging, the two automatic tracks, and the rule that an ablation which does not help gets reported anyway. We wrote the relevance judgments ourselves; no model graded its own output. We reviewed the generated code, ran it on the real corpus rather than on stubs, and are answerable for every claim in this report.

What the AI did not do

It did not choose the architecture, the corpus policy or the evaluation design. It did not produce the relevance judgments. It did not produce any number in §5 — those come from running the harness, and until they do they are visible placeholders.

Tooling

Claude Code (Anthropic) for implementation, tests, the video project and report drafting. Standard libraries only for the IR core: NumPy for the postings arrays and the power iteration, pandas and PyArrow for the metadata, a PDF text extractor for ingest. No search engine, no graph library, no embedding model, no ranking library — the index, the merges, the scorers and the PageRank are ours.

References

1. C. D. Manning, P. Raghavan & H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. — SPIMI, skip pointers, lnc.ltc, zone scoring, champion lists, tiered indexes, index elimination.
2. S. Robertson & H. Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval 3(4), 2009. — BM25 and the BM25F field-combination rule.
3. L. Page, S. Brin, R. Motwani & T. Winograd. *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab, 1999.
4. K. Järvelin & J. Kekäläinen. *Cumulated Gain-based Evaluation of IR Techniques*. ACM TOIS 20(4), 2002. — nDCG.
5. M. F. Porter. *An Algorithm for Suffix Stripping*. Program 14(3), 1980.
6. Indian Supreme Court Judgments. AWS Open Data Registry, CC-BY-4.0.
7. The Bharatiya Nyaya Sanhita, 2023 (in force 1 July 2024). — the IPC↔BNS section mapping.

Appendix A — reproducing the results

```
python -m nyaya.build --download --extract --index     # corpus and S1-S5
python -m pytest tests -q                              # all component tests
python -m nyaya.cli --smoke                            # end-to-end smoke test
python -m nyaya.eval.run --systems B0,S1,S2,S2p,S3,S4 --qrels eval/queries
python -m nyaya.eval.operators --brute-force
python -m nyaya.eval.recovery
python -m nyaya.eval.efficiency
```

Outputs land in `runs/` as CSV, JSON and PNG. `config.SAMPLE_SEED = 358` fixes the corpus draw, so a rebuild reproduces the same indexed subset.

Appendix B — query language reference

Form	Meaning
`murder bail`	free text; fast path through tier 1
`"common intention"`	exact phrase
`privacy /5 dignity`	within five words
`privacy /s dignity`	same sentence
`privacy /p dignity`	same paragraph
`bail!`	root expansion by dictionary prefix
`title:(privacy)`	restrict to one zone
`year:>=2000`, `bench:>=5`	parametric filters
`judge:chandrachud`, `act:IPC`, `type:criminal`	parametric filters
`X AND Y`, `X OR Y`, `X NOT Y`	Boolean; `NOT` is a set difference, never a bare negation